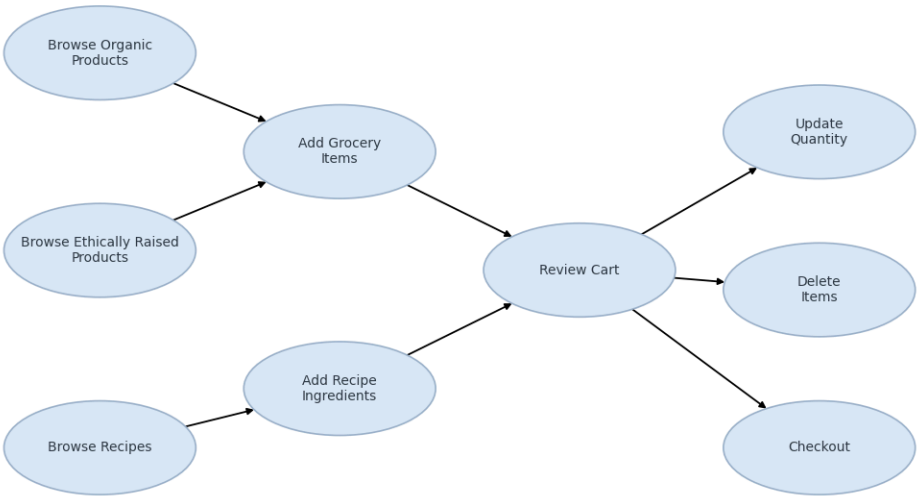
Project: FreshRoute

Sprint 2 Duration: March 10 - March 23, 2026

Network Diagram

The visual representation of task dependencies is attached below.



Critical Path

The critical path determines the main dependency chain that had the greatest impact on whether the sprint could be completed on time. For FreshRoute Sprint 2, the critical path was:

1. Browse Organic Products / Browse Ethically Raised Products
2. Add Grocery Items
3. Review Cart
4. Checkout

This path was treated as the most important sequence because it represents the core shopping flow of the application. A user must first browse products, then add items to the cart, review the cart contents, and finally proceed to checkout. The recipe flow was also important, but it acted as a supporting branch that fed into the cart rather than replacing the main checkout path.

Strategies to Stay on Schedule

Task Prioritization: We first focused on building the main product-related pages, including the Organic, Ethically Raised, and Recipes pages. After those core pages were completed, we moved on to the cart-related functionality, including adding items to the cart and developing the cart page. This helped us build the main shopping flow in a clear order.

Parallel Development: Different team members worked on filtering, recipe functionality, organic and ethical pages, cart features, documentation, populating database and backend integration at the same time so progress could continue across multiple areas of the sprint.

GitHub Branching: We used separate feature branches for cart updates, recipe-related work, filtering, organic and ethical features and backend integration so changes could be developed independently and merged in a more organized way.

Standups and Check-ins: Regular standups helped the team identify blockers early, communicate progress, and reassign effort when needed.

Incremental Integration: We built the user-facing pages first and then connected supporting backend and data integration work step by step, which helped reduce risk during the sprint

---

What Went Wrong

One challenge during the sprint was that several tasks depended on shared cart behavior and integration logic. Because both grocery products and recipe ingredients eventually flowed into the cart, changes in one part of the system sometimes affected another part and required extra testing. Another issue was that backend integration required more coordination than expected, especially when connecting product data, recipe data, and images with the frontend pages. This slowed some parts of the workflow and made debugging more time-consuming than initially planned.

Lessons Learned

Identify the main user flow early: Defining the cart and checkout flow as the most important path helped the team protect the most valuable work during the sprint.

Start integration-heavy work sooner: Features that depend on shared data or shared cart logic should begin earlier so there is more time for testing and debugging.

Keep tasks at the right level of detail: For schedule planning, it is better to focus on major dependent tasks such as browsing, adding items, reviewing the cart, and checkout instead of low-level implementation details.

Parallel work must still be coordinated carefully: Even when team members work on different branches, related features still need frequent communication so integration is smoother later.